

INGENIERÍA DE SOFTWARE



AYUNDANTÍA UNIDAD 2

INFO248

Universidad Austral de Chile

2026

ACTIVIDAD

Mira, la idea es montar un museo de informática dentro del edificio 10K, pero no algo estático, sino una experiencia interactiva. Queremos que las personas puedan recorrer el hall con las vitrinas y, al lado de cada objeto, escaneen un código QR con su celular que los lleve a una página donde se muestre información del objeto, idealmente con una sección donde la gente pueda comentar (para compartir si tuvieron alguna experiencia de trabajo real con el objeto presentado), además de un video o audio explicativo; esa misma página debería organizar bien lo que se muestra, porque no todos los visitantes van a interactuar igual (algunos solo miran, otros quieren explorar más).

Desde ahí también debería existir la opción de pasar a otra aplicación donde puedan ver ese mismo objeto en realidad aumentada, como si estuviera frente a ellos, y ojalá interactuar un poco (por ejemplo, encender un computador antiguo o ver un robot moverse alrededor). Por otro lado, queremos algo más interno: una aplicación que nos permita cargar y procesar este tipo de contenido del museo (modelos 3D, audios, comentarios), donde ese contenido pase por distintas etapas antes de quedar disponible para los visitantes, como validaciones, conversiones de formato y almacenamiento final. Esto con el fin de verificar que la experiencia que se muestra a los visitantes sea de buena calidad.

Además, necesitamos una aplicación de administración donde el equipo del museo pueda gestionar los objetos, actualizar información y trabajar la información recopilada para obtener estadísticas básicas de uso (qué objetos son más visitados; visitas por hora del día). Todas estas aplicaciones deberían comunicarse entre sí cuando sea necesario, pero sin que dependan demasiado unas de otras, porque la idea es poder ir mejorando partes sin tener que rehacer todo. Además, se debe habilitar la opción de subir documentos antiguos con información de los objetos. Esta información no se ingresará manualmente, debe ser automatizado para simplemente subir imágenes de estos y que el sistema extraiga la información relevante, corrija errores, la organice de manera adecuada y la deje disponible. Esto es importante para garantizar que el museo pueda crecer en contenido sin requerir un esfuerzo manual excesivo, permitiendo así una escalabilidad eficiente a medida que se añadan más objetos y visitantes al museo. ordenado para que escale si el museo crece en contenido o visitantes.

SE SOLICITA

- ¿Qué sistemas identifican? ¿Qué arquitecturas son apropiadas a cada uno?
- ¿Conviene SQRC para la BDD y cómo se vería reflejada?
- Hacer el diagrama de componentes de cada sistema y un diagrama de secuencia por cada uno.
- ¿Alguna solución para cambiar a microservicios? ¿Es viable?

SISTEMAS Y ARQUITECTURAS

APLICACIÓN DE REALIDAD AUMENTADA → 3 CAPAS

- Se separa la interfaz AR, la lógica de interacción y el acceso a modelos e información.

GESTIÓN DE DOCUMENTOS FÍSICOS → PIPE AND FILTER

- Los documentos pasan por distintas etapas, como recepción, interpretación, conversión y almacenamiento.

SUBIDA Y PROCESAMIENTO DE MODELOS 3D → PIPE AND FILTER

- Los modelos deben validarse, convertirse y optimizarse antes de quedar disponibles.

GESTIÓN DE INFORMACIÓN DE OBJETOS → CLIENTE-SERVIDOR

- Los administradores gestionan la información desde clientes conectados a un servidor central con acceso a los datos.

VISTA DE INFORMACIÓN DE OBJETOS → MVC

- Se separa la vista mostrada al usuario, el control de navegación y la gestión de datos del objeto.

¿CONVIENE CQRS?

Sí, porque el sistema hace muchas más lecturas que escrituras.

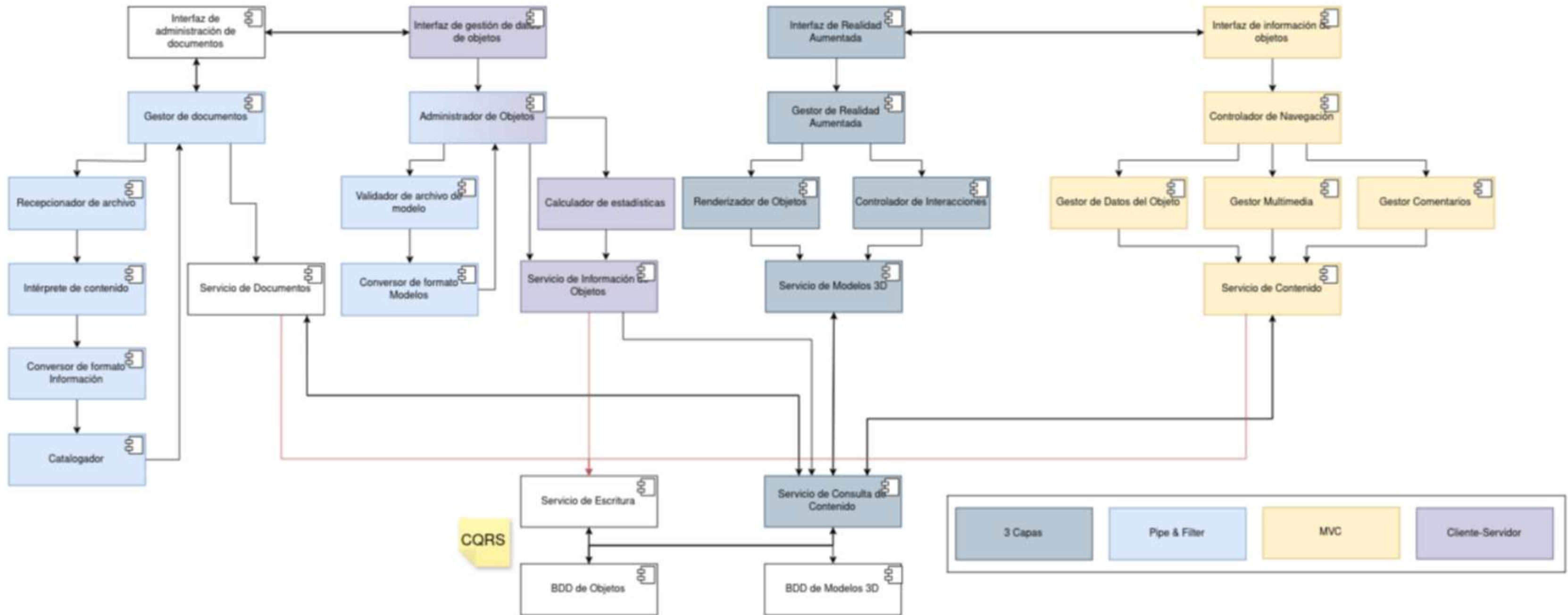
- La mayoría de usuarios solo consulta información, mientras que pocos administradores modifican contenido. Por esto conviene separar lectura y escritura.

¿CONVIENE USAR MICROSERVICIOS?

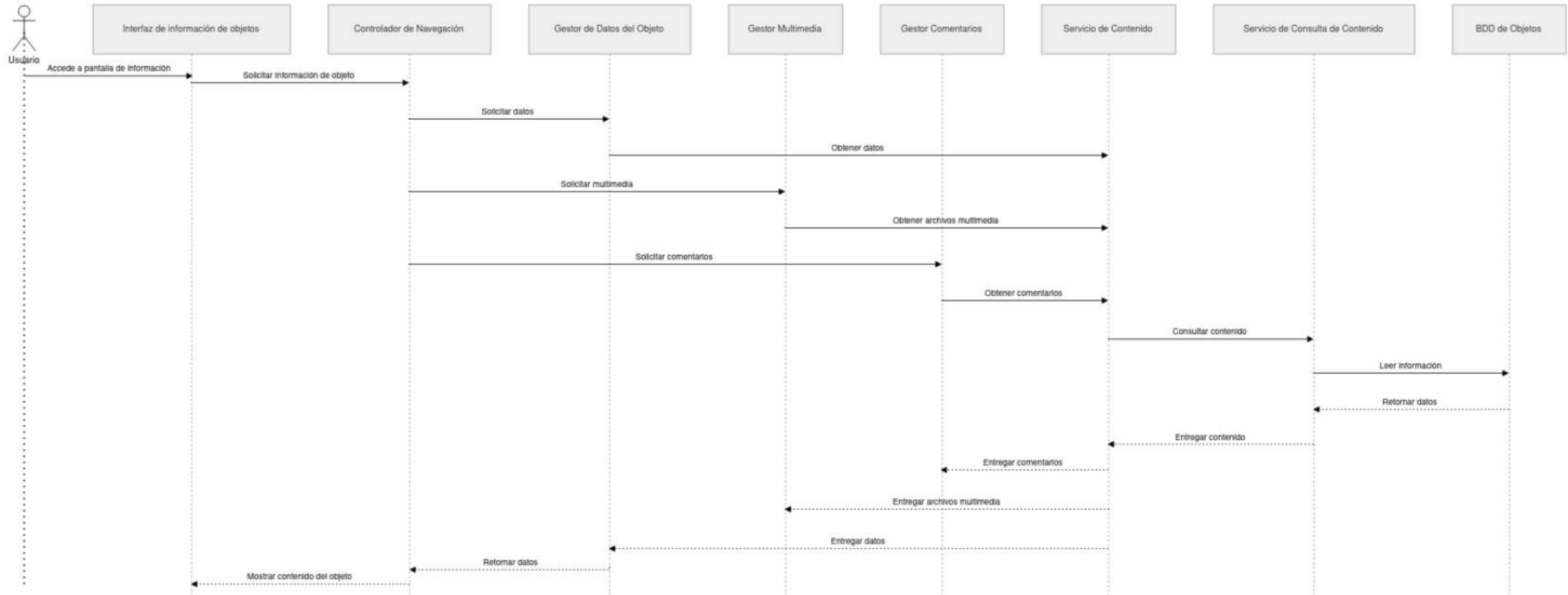
No mucho.

- Todos los sistemas usan prácticamente la misma información, así que separarlos en microservicios obligaría a duplicar datos y sincronizarlos constantemente, aumentando la complejidad innecesariamente.
- Una solución modular centralizada es más simple y eficiente para este caso.

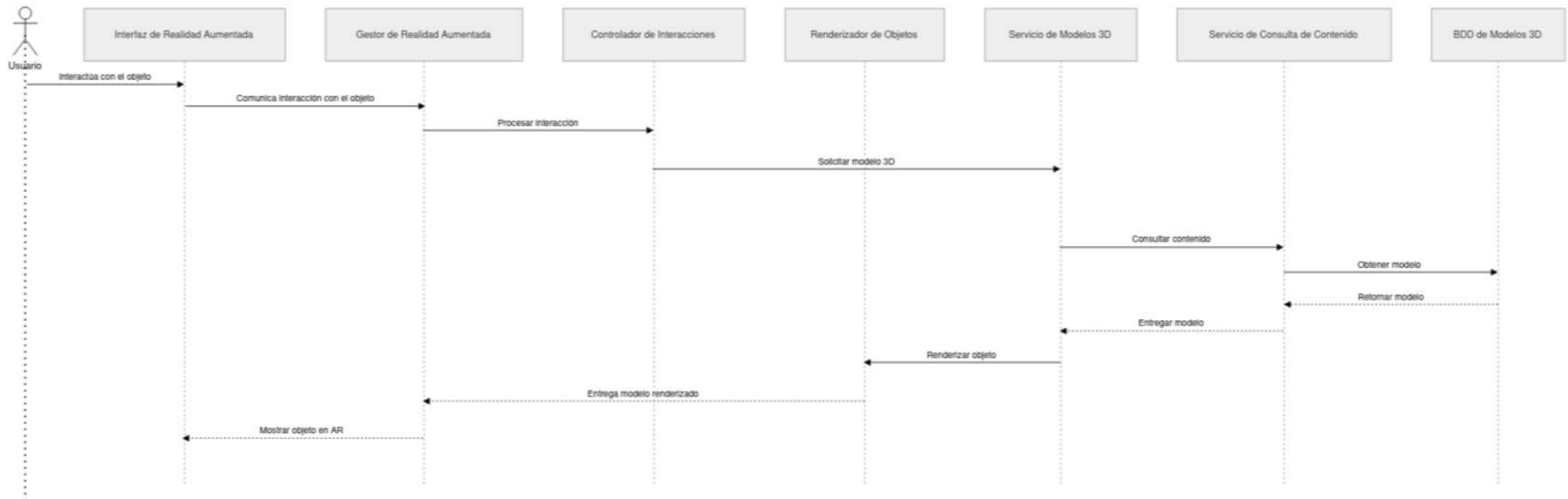
COMPONENTES



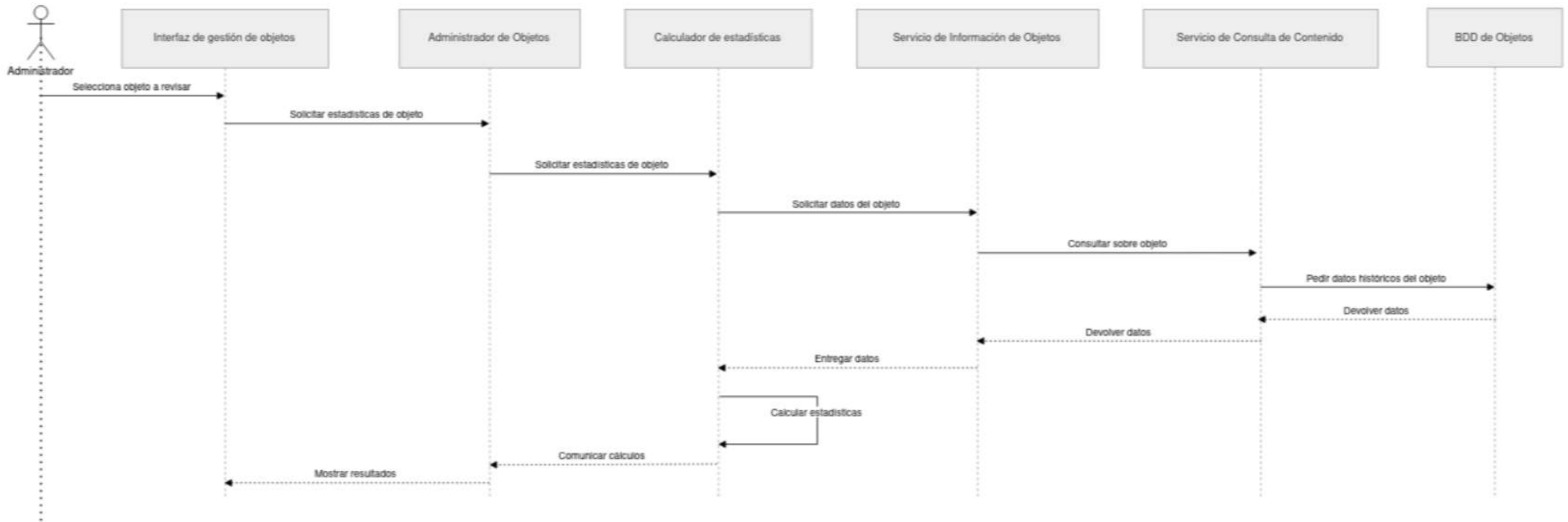
VER INFORMACIÓN DE UN OBJETO



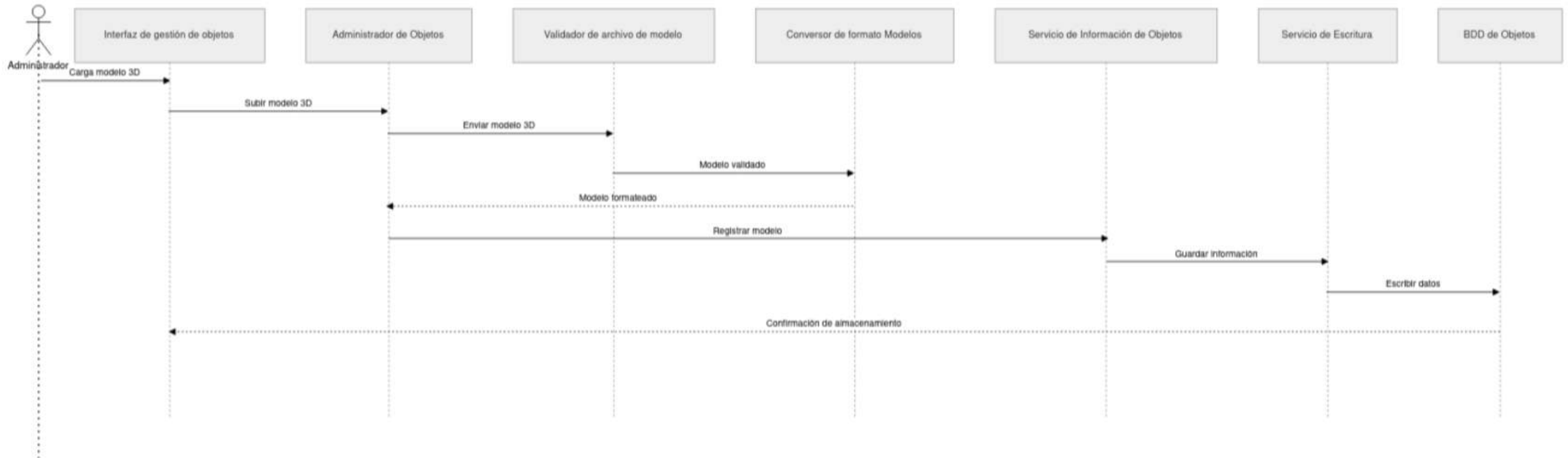
INTERACTUAR CON UN OBJETO EN REALIDAD AUMENTADA



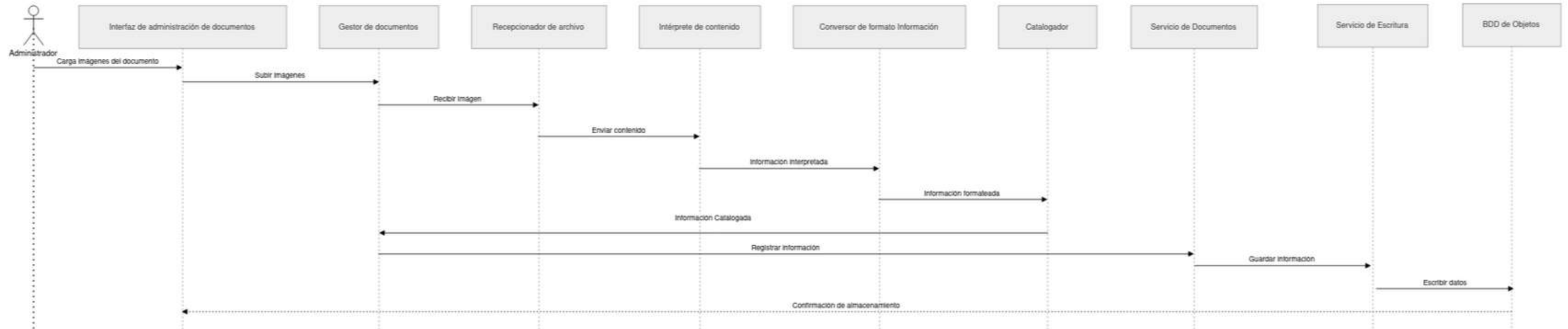
VER ESTADÍSTICAS DE UN OBJETO



SUBIR UN MODELO 3D PARA UN OBJETO



CARGAR UN DOCUMENTO FÍSICO



REFACTORING

“ La refactorización (refactoring) es el proceso de reorganizar el código existente para mejorar su estructura interna sin alterar su comportamiento externo. Su propósito es hacer el software más comprensible, mantenible y económico de modificar, reduciendo la deuda técnica y evolucionando el sistema hacia un diseño más limpio y simple. ”

Refactoring.com



REFACTORING

1

CLASE GRANDE

2

CÓDIGO DUPLICADO

3

MÉTODO LARGO (MAIN)

4

OBSESIÓN PRIMITIVA

5

NIVELES DE ABSTRACCIÓN HETEROGÉNEOS

6

ENCAPSULAMIENTO DEFICIENTE

1) CLASE GRANDE

Separación de responsabilidades: el usuario conserva sus datos y la cartera maneja el dinero.

Antes

Clase monolítica

```
class User:
    def __init__(self, name, email, address, credit_card, pin, billing_info):
        self.name = name
        self.email = email
        self.address = address
        self.credit_card = credit_card
        self.pin = pin
        self.billing_info = billing_info
        self.wallet_balance = 0
        self.rentals = []
        self.purchases = []
        self.notifications = []
```

Después

Responsabilidad separada

```
class Wallet:
    def __init__(self):
        self.balance = 0

    def credit(self, amount):
        self.balance += amount

    def debit(self, amount):
        if self.balance < amount:
            raise ValueError("Insufficient balance")
        self.balance -= amount

class User:
    def __init__(self, name, email):
        self.name = name
        self.email = email
        self.wallet = Wallet()
```

Referencia: [extractClass](#)

2) CÓDIGO DUPLICADO

Centralización del pago para evitar repetir la misma validación en cada operación.

Antes

Lógica repetida

```
def register_rental(self, movie):
    if self.wallet_balance >= movie.rental_price:
        self.wallet_balance -= movie.rental_price
        ...
    else:
        print("Insufficient balance")

def register_purchase(self, movie):
    if self.wallet_balance >= movie.purchase_price:
        self.wallet_balance -= movie.purchase_price
        ...
    else:
        print("Insufficient balance")
```

Después

Un solo punto de pago

```
class PaymentProcessor:
    def process(self, wallet, amount):
        wallet.debit(amount)

def register_rental(self, movie, payment_processor):
    payment_processor.process(self.wallet, movie.rental_price)

def register_purchase(self, movie, payment_processor):
    payment_processor.process(self.wallet, movie.purchase_price)
```

Referencia: [replaceInlineCodeWithFunctionCall](#) (*)

3) MÉTODO LARGO (MAIN)

Separación por casos de uso: preparar usuario y procesar transacciones en funciones distintas.

Antes

Flujo concentrado

```
def main():
    user = User(...)
    movie1 = Movie("Inception", 3.99, 14.99)
    movie2 = Movie("The Matrix", 2.99, 12.99)

    user.add_money_to_wallet(20)
    user.register_rental(movie1)
    user.register_purchase(movie2)

    user.get_rentals()
    user.get_purchases()
```

Después

Funciones por intención

```
def setup_user():
    user = User("John Doe", "john@example.com")
    user.wallet.credit(20)
    return user

def process_transactions(user):
    movie1 = Movie("Inception", 3.99, 14.99)
    movie2 = Movie("The Matrix", 2.99, 12.99)

    payment = PaymentProcessor()

    user.register_rental(movie1, payment)
    user.register_purchase(movie2, payment)

def main():
    user = setup_user()
    process_transactions(user)
```

Referencia: [splitPhase](#)

4) OBSESIÓN PRIMITIVA

Modelado explícito para reemplazar estructuras anónimas con un objeto de dominio claro.

Antes

Diccionario ad hoc

```
rental = {  
    'movie': movie,  
    'start_date': datetime.datetime.now(),  
    'end_date': datetime.datetime.now() + datetime.timedelta(days=30),  
    'views_remaining': 1  
}
```

Después

Objeto de dominio

```
class Rental:  
    def __init__(self, movie):  
        self.movie = movie  
        self.start_date = datetime.datetime.now()  
        self.end_date = self.start_date + datetime.timedelta(days=30)  
        self.views_remaining = 1  
  
rental = Rental(movie)  
self.rentals.append(rental)
```

Referencia: [replacePrimitiveWithObject](#)

5) NIVELES DE ABSTRACCIÓN HETEROGÉNEOS

Separación en servicio de dominio para no mezclar reglas de negocio con detalles técnicos.

Antes

Mezcla de responsabilidades

```
def register_rental(self, movie):  
    if self.wallet_balance >= movie.rental_price:  
        self.wallet_balance -= movie.rental_price  
        rental = {...}  
        self.rentals.append(rental)
```

Después

Servicio de dominio

```
class RentalService:  
    def rent(self, user, movie, payment_processor):  
        payment_processor.process(user.wallet, movie.rental_price)  
        rental = Rental(movie)  
        return rental  
  
rental = rental_service.rent(user, movie, payment)  
user.rentals.append(rental)
```

Referencia: [extractFunction](#)

6) ENCAPSULAMIENTO DEFICIENTE

La lógica de pago debe vivir en el comportamiento del objeto, no en campos expuestos directamente.

Antes

Acceso directo al estado

```
class PaymentProcessor:
    def process_payment(self, user, amount):
        if user.wallet_balance >= amount:
            user.wallet_balance -= amount
        return True
```

Después

Encapsulación por comportamiento

```
class User:
    def pay(self, amount):
        self.wallet.debit(amount)

class PaymentProcessor:
    def process(self, user, amount):
        user.pay(amount)
```

Referencia: [encapsulateVariable](#), [hideDelegate](#)

CLASIFICACIÓN POR ACCESIBILIDAD



```
]

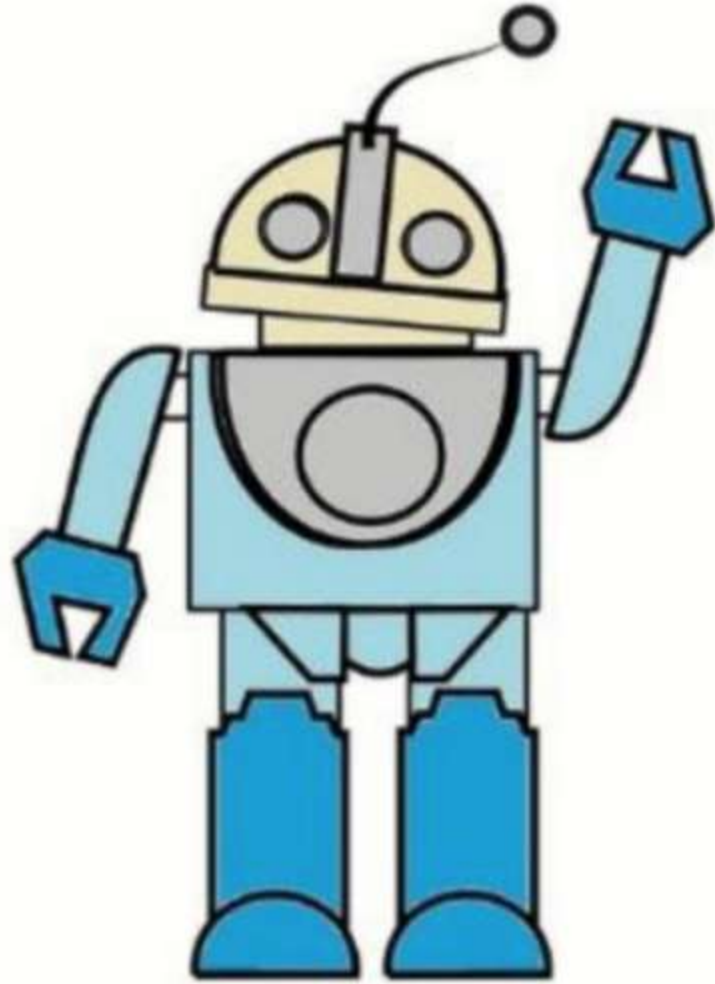
def test_endpoint(endpoint):
    url = BASE_URL + endpoint
    try:
        r = requests.get(url, timeout=3)
        print(f"[GET] {url} -> {r.status_code}")
    except Exception as e:
        print(f"[ERROR] {url} -> {e}")

def test_login(endpoint):
    url = BASE_URL + endpoint
    for payload in payloads:
        try:
            r = requests.post(url, json=payload, timeout=3)
            print(f"[POST] {url} {payload} -> {r.status_code}")
        except Exception as e:
            print(f"[ERROR] {url} -> {e}")

if __name__ == "__main__":
    for ep in endpoints:
        test_endpoint(ep)

    test_login("/login")
```

CLASIFICACIÓN POR EJECUCIÓN



**Pruebas
Automatizadas**



**Pruebas
semiautomáticas**



Pruebas Manuales


```
print("\nPayloads actuales:")
for i, p in enumerate(payloads):
    print(f"{i}: {p}")

edit = input("\n¿Quieres añadir un payload? (y/n): ").lower()
if edit == "y":
    u = input("username: ")
    p = input("password: ")
    payloads.append({"username": u, "password": p})

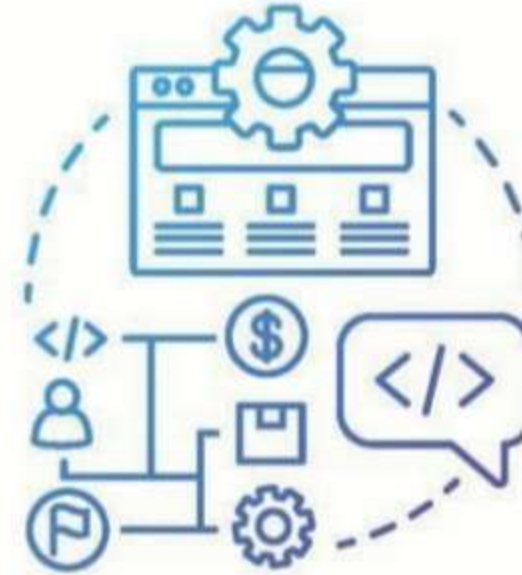
def test_get(url):
    try:
        r = requests.get(url, timeout=3)
        print(f"[GET] {url} -> {r.status_code}")
    except Exception as e:
        print(f"[ERROR] {url} -> {e}")

def test_post(url, payload):
    try:
        r = requests.post(url, json=payload, timeout=3)
        print(f"[POST] {url} {payload} -> {r.status_code}")
    except Exception as e:
        print(f"[ERROR] {url} -> {e}")
```


CLASIFICACIÓN SEGÚN ALCANCE



Pruebas Funcionales



Pruebas No-Funcionales

CLASIFICACIÓN POR ALCANCE: TIPOS

PRUEBAS FUNCIONALES

- UAT
- Exploratorias
- Humo
- Sanidad
- Regresión
- Etc.

PRUEBAS NO-FUNCIONALES

- Rendimiento
 - Carga
 - Estrés
 - Spike
 - Resistencia
 - Volumen
 - Escalabilidad
- Seguridad
- Usabilidad
- Compatibilidad
- Etc.

FORMATO DE CASO DE PRUEBA

SUT → componente específico (no todo el sistema)

Input → datos concretos (evitar ambigüedad)

Setup → estado inicial controlado

Invocation → pasos reproducibles

Expected Result → comportamiento esperado

Oracle → cómo validas objetivamente (logs, DB, UI)

Teardown → limpieza (clave en automatización)

EN BASE AL CASO DEL MUSEO INTERACTIVO, DISEÑE:

- Un (1) caso de prueba funcional.
- Dos (2) casos de prueba no funcionales (de distintas categorías).

ADEMÁS, CLASIFIQUE CADA CASO SEGÚN:

- Tipo de ejecución
- Accesibilidad
- Tipo de prueba específica



Caso de Prueba: Visualización de contenido desde QR · **Tipo:** Funcional

Software bajo prueba (SUT): Página de información de objetos

Datos de prueba (Input): QR válido asociado a objeto "Computador IBM 1980"

SETUP

- Objeto existente en base de datos
- Contenido (texto, video) previamente cargado
- Servidor activo

INVOCATION

1. Escanear QR con dispositivo móvil
2. Acceder a URL generada
3. Esperar carga de la página

RESULTADO ESPERADO

- Se despliega información del objeto
- Se muestra video/audio asociado
- Se habilita sección de comentarios

Oracle: Manual, Martín Maza

Teardown: Limpiar caché del navegador. Eliminar logs de acceso al sistema.

CLASIFICACIÓN

Ejecución: Manual · Accesibilidad: Black Box · Tipo: Prueba funcional (UAT / humo)

Caso de Prueba: Carga concurrente de accesos QR · **Tipo:** No Funcional

Software bajo prueba (SUT): Backend + Página de información de objetos

Datos de prueba (Input): 500 instancias simultáneas accediendo a distintos QRs

SETUP

- Herramienta de carga (JMeter) configurada con 500 requests
- Base de datos con 100 objetos
- Servidor activo
- Interfaz de JMeter lista para monitoreo

INVOCATION

1. Iniciar la ejecución del plan de pruebas en JMeter
2. Mantener la carga durante 5 minutos
3. Detener la ejecución del plan de pruebas

RESULTADO ESPERADO

- Tiempo de respuesta < 2 segundos promedio
- Tasa de error < 1%

Oracle: Automatizada, JMeter

Teardown: Liberar recursos. Eliminar logs.

CLASIFICACIÓN

Ejecución: Automatizada · Accesibilidad: Black Box · Tipo: Rendimiento → Carga

Caso de Prueba: Procesamiento masivo de documentos físicos · **Tipo:** No Funcional

Software bajo prueba (SUT): Pipeline de procesamiento de documentos físicos

Datos de prueba (Input): 1000 imágenes de documentos históricos

SETUP

- Sistema de colas activo
- Almacenamiento disponible
- Servidor activo

INVOCATION

1. Acceder a la interfaz de carga de documentos
2. Pulsa el botón de subir documentos
3. Cargar lote de imágenes
4. Pulsar el botón de procesar

RESULTADO ESPERADO

- $\geq 95\%$ documentos procesados completamente
- Tiempo de ejecución total proporcional a la cantidad de documentos
- No hay pérdida de datos

Oracle: Manual, Martín Maza

Teardown: Eliminar datos de prueba. Resetear colas. Eliminar logs.

CLASIFICACIÓN

Ejecución: Manual · Accesibilidad: Black Box · Tipo: Rendimiento → Escalabilidad / Volumen